

UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA

Facultad de Ingeniería

Ingeniería de Sistemas

INFORME DE LABORATORIO 4

AMQP y tercer protocolo: comparación con MQTT hacia Azure IoT Central

Presentado por:

Juan Sebastian Lizcano Jaimes

Angie Katherine Suarez Ortiz

Miguel Angel Hernandez Quintero

IoT + Cloud + Sistemas Distribuidos

Bucaramanga, octubre de 2026

Resumen ejecutivo

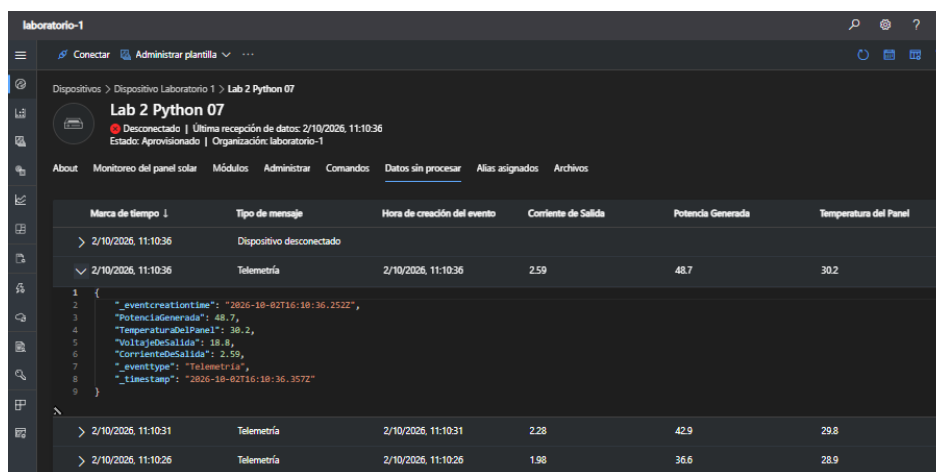
Se cambió el transporte del dispositivo Python de la VM a AMQP, se agregó HTTPS como tercer protocolo y se compararon ambos con MQTT, la línea base del Laboratorio 3. Los tres clientes publicaron las mismas cuatro variables del panel solar en Lab 2 Python 07, dentro de la aplicación laboratorio-1, y se midieron con el mismo método: bytes de la conexión TCP, tiempo de confirmación y tiempo entre la creación del mensaje y su registro en IoT Hub. Las capturas están en el Anexo A.

1. Rol de AMQP en Azure

AMQP 1.0 organiza el intercambio en conexiones, sesiones y enlaces, y controla el flujo con créditos: el receptor indica cuántos mensajes acepta antes de recibirlos. Cada entrega termina con un estado propio (accepted, rejected o released), lo que permite reintentar exactamente lo que falló. Por eso Azure lo usa en el plano de servicio: Service Bus y Event Hubs lo hablan de forma nativa, y el endpoint de eventos de IoT Hub, compatible con Event Hubs, se consume por AMQP. Para el dispositivo, IoT Hub acepta AMQP en el puerto 5671 o sobre WebSockets en el 443; desde la VM el 5671 estaba abierto, así que se usó AMQPS directo.

2. AMQP hacia IoT Central

Como azure-iot-device para Python solo trae MQTT, el cliente `amqp_cliente.py` se hizo con Qpid Proton 0.40. Se conecta al hub de la aplicación por el puerto 5671 con TLS 1.2, se autentica con SASL PLAIN (usuario `lab2-python-07@sas.{hub}` y token SAS como contraseña) y abre una sesión con un enlace de envío hacia `/devices/lab2-python-07/messages/events`. IoT Hub concedió 50 créditos y aceptó los 10 mensajes en 133 a 171 ms. En Datos sin procesar aparecen las cuatro variables con su hora de creación:



Marca de tiempo	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del Panel
> 2/10/2026, 11:10:36	Dispositivo desconectado				
< 2/10/2026, 11:10:36	Telemetría	2/10/2026, 11:10:36	2.59	48.7	30.2
<pre>1 { 2 "_eventcreationTime": "2026-10-02T16:10:36.252Z", 3 "PotenciaGenerada": 48.7, 4 "TemperaturaDelPanel": 30.2, 5 "VoltajeDeSalida": 18.8, 6 "CorrienteDeSalida": 2.59, 7 "eventType": "Telemetria", 8 "_timestamp": "2026-10-02T16:10:36.357Z" 9 }</pre>					
> 2/10/2026, 11:10:31	Telemetría	2/10/2026, 11:10:31	2.28	42.9	29.8
> 2/10/2026, 11:10:26	Telemetría	2/10/2026, 11:10:26	1.98	36.6	28.9

Figura 1. Mensaje AMQP con las cuatro variables en Datos sin procesar de IoT Central.

Frente al MQTT del Lab 3, la conexión también es persistente, pero la sesión y el enlace son explícitos, cada entrega tiene su estado y la traza de tramas permite depurar el protocolo sin descifrar TLS. El costo es un establecimiento más pesado (285 ms y 1216/4200 bytes) y unos 74 bytes más por mensaje. AMQP resuelve mejor que MQTT el control de flujo y la confirmación individual entre servicios: el crédito evita saturar al receptor y el estado de cada entrega permite reintentos precisos.

3. Tercer protocolo: HTTPS

Se eligió HTTPS por una razón de negocio: varios inversores y dataloggers de plantas solares solo tienen salida por el puerto 443 según la política de red del cliente, y reportan cada 5 a 15 minutos, así que mantener una sesión abierta no aporta. `https_cliente.py` hace POST a `/devices/lab2-python-07/messages/events` con el token SAS en el encabezado Authorization; IoT Hub respondió 204 en los 20 envíos y los datos llegaron a la misma aplicación. Ventajas: atraviesa firewalls y proxies, se depura con cualquier herramienta HTTP y no necesita librerías nuevas. Desventajas: cada petición repite encabezados y token (766 bytes de subida, casi el triple que MQTT), con conexión nueva el saludo TLS suma unos 5 KB por mensaje y no admite comandos en línea, solo consulta de mensajes. La implementación real fueron unas 150 líneas sin dependencias nuevas; lo delicado fue armar el token y los encabezados `iothub-*`.

4. Tabla comparativa MQTT, AMQP y HTTPS

Criterio	MQTT	AMQP	HTTPS
Overhead por mensaje	263 bytes de subida y 33 de bajada	337 bytes de subida y 55 de bajada	766 de subida y 93 de bajada; 1379 y 3617 con TLS nuevo
Establecimiento	203 ms; 963/3620 bytes (TLS y CONNECT)	285 ms; 1216/4200 bytes (TLS, SASL, open, begin y attach)	129 ms; 611/3524 bytes (solo TLS)
Latencia percibida	PUBACK en 141 ms; creación a IoT Hub 107 ms	Disposición en 142 ms; creación a IoT Hub 109 ms	Respuesta en 144 ms (188 con TLS nuevo); creación a IoT Hub 114 ms
Fiabilidad	QoS 0 y 1; PUBACK por mensaje; la sesión resiste cortes cortos	Estado por entrega, crédito y modos de liquidación	Código HTTP por petición; reintentos a cargo del cliente
Soporte en microcontrolador / Wokwi	Nativo en el ESP32 con <code>esp-mqtt</code> y Azure SDK for C	Poco práctico: el SDK embebido de Azure no trae AMQP	Posible con <code>HttpClient</code> y TLS, pero caro por mensaje
Facilidad de debug	Topics y QoS legibles en los registros	Traza de tramas con <code>PN_TRACE_FRM</code>	La más simple: curl, encabezados y códigos de estado
Paso de firewalls	8883 suele bloquearse; MQTT sobre WebSockets en 443	5671 suele bloquearse; AMQP sobre WebSockets en 443	El mejor: 443 y compatible con proxies
Caso de uso ideal	Telemetría frecuente desde dispositivos	Servicios y gateways con mucho volumen	Envíos esporádicos desde redes restringidas

5. Recomendaciones por capa

- **Dispositivo (Wokwi y ESP32 futuro):** MQTT con QoS 1 sobre TLS y, si la red bloquea el 8883, MQTT sobre WebSockets en el 443. Es el más liviano por mensaje y el único que trae el SDK embebido de Azure.
- **Entre servicios Azure y en la plataforma propia (Labs 5 a 8):** AMQP, porque Service Bus y Event Hubs lo usan de forma nativa y el crédito y el estado por entrega sirven para colas y procesamiento con garantías.
- **Dos protocolos para el resto del curso:** MQTT en el borde y AMQP en el backend. HTTPS queda para integraciones puntuales o equipos detrás de proxies, porque cuesta casi el triple por mensaje y no recibe comandos en línea.

Anexo A. Evidencias del procedimiento

Las capturas siguen el orden en que se hicieron las pruebas el 2 de octubre de 2026. Las horas de IoT Central están en hora de Colombia y las de los registros en UTC.

A.1 Entorno y preparación

Se trabajó en la VM del Laboratorio 2 (Debian 12 y Python 3.11), en la carpeta ~/laboratorio4 con su propio entorno virtual. Se instalaron libssl-dev y pkg-config para compilar Qpid Proton 0.40 con soporte TLS, y con pip se instalaron python-qpid-proton, paho-mqtt, python-dotenv y certifi. El archivo .env reutiliza el hostname del hub, el Device ID y la clave de lab2-python-07 del Laboratorio 3 y no se sube al repositorio. Los tres clientes comparten el módulo comun.py: el mismo generador de lecturas, el mismo token SAS y la misma forma de medir, con los bytes de cada conexión tomados con ss y los tiempos con el reloj de la VM. El ESP32 de Wokwi sigue disponible como testigo MQTT de los laboratorios anteriores.

A.2 Puertos y TLS

El hub de la aplicación resolvió a 20.44.8.231, un gateway de IoT Hub en Central US, y los puertos 8883, 5671 y 443 respondieron. El saludo TLS del 5671 negoció TLS 1.2 con ECDHE-RSA-AES128-GCM-SHA256 y certificado válido, así que no hizo falta AMQP sobre WebSockets.

```
(.venv) lab2@Laboratorio-2:~/Laboratorio4$ ./verificar_puertos.sh
Hub: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net
20.44.8.231 gateway-prod-gw-centralus-6-g2.centralus.cloudapp.azure.com iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net

Connection to iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net (20.44.8.231) 8883 port [tcp/*] succeeded!
Connection to iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net (20.44.8.231) 5671 port [tcp/amqps] succeeded!
Connection to iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net (20.44.8.231) 443 port [tcp/https] succeeded!

Saludo TLS en el puerto 5671 (AMQPS):
New, TLSv1.2, Cipher is ECDHE-RSA-AES128-GCM-SHA256
Protocol : TLSv1.2
Verify return code: 0 (ok)
(.venv) lab2@Laboratorio-2:~/Laboratorio4$ |
```

Figura A1. Resolución del hub, puertos abiertos y TLS del puerto 5671.

A.3 AMQP hacia IoT Central

La corrida formal envió 10 mensajes cada 5 s. El encabezado resume la sesión: puerto 5671, TLS validado, SASL PLAIN con token SAS y destino del enlace. La conexión, la sesión y el enlace quedaron listos en 285 ms y IoT Hub concedió 50 créditos; cada transferencia descuenta uno.

```
(.venv) lab2@Laboratorio-2:~/laboratorio4$ python amqp_cliente.py --count 10 --interval 5
===== AMQP =====
Hostname: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net
Puerto: 5671 (amqps)
TLS: Sí, certificado validado con certifi
Autenticación: SASL PLAIN con token SAS
Usuario SASL: lab2-python-07@sas.iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93
Destino del enlace: /devices/lab2-python-07/messages/events
Clave: Cargada, no mostrada
=====
[CONNECT] Abriendo conexión AMQP, sesión y enlace de envío...
[AMQP] Conexión abierta con iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net
[AMQP] Sesión y enlace listos en 285.13 ms
[AMQP] Bytes de establecimiento (TLS + SASL + open/begin/attach): enviados=1216 recibidos=4200
[FLOW] Crédito concedido por IoT Hub: 50 mensajes

[TRANSFER] secuencia=0
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 100
[TRANSFER] payload= {"PotenciaGenerada":18.1,"TemperaturaDelPanel":26.4,"VoltajeDeSalida":17.9,"CorrienteDeSalida":1.01}
[TRANSFER] crédito restante= 49
[DISPOSITION] secuencia=0 estado=accepted tiempo=170.54 ms

[TRANSFER] secuencia=1
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 100
[TRANSFER] payload= {"PotenciaGenerada":19.0,"TemperaturaDelPanel":26.4,"VoltajeDeSalida":17.9,"CorrienteDeSalida":1.06}
[TRANSFER] crédito restante= 48
[DISPOSITION] secuencia=1 estado=accepted tiempo=136.33 ms

[TRANSFER] secuencia=2
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 99
[TRANSFER] payload= {"PotenciaGenerada":19.8,"TemperaturaDelPanel":26.7,"VoltajeDeSalida":18.0,"CorrienteDeSalida":1.1}
[TRANSFER] crédito restante= 47
[DISPOSITION] secuencia=2 estado=accepted tiempo=143.25 ms
|
```

Figura A2. Inicio de la corrida AMQP: parámetros, establecimiento, crédito y primeras entregas.

```
[TRANSFER] secuencia=5
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 100
[TRANSFER] payload= {"PotenciaGenerada":28.0,"TemperaturaDelPanel":27.9,"VoltajeDeSalida":18.3,"CorrienteDeSalida":1.53}
[TRANSFER] crédito restante= 44
[DISPOSITION] secuencia=5 estado=accepted tiempo=146.44 ms

[TRANSFER] secuencia=6
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 99
[TRANSFER] payload= {"PotenciaGenerada":31.3,"TemperaturaDelPanel":28.2,"VoltajeDeSalida":18.4,"CorrienteDeSalida":1.7}
[TRANSFER] crédito restante= 43
[DISPOSITION] secuencia=6 estado=accepted tiempo=134.24 ms

[TRANSFER] secuencia=7
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 100
[TRANSFER] payload= {"PotenciaGenerada":36.6,"TemperaturaDelPanel":28.9,"VoltajeDeSalida":18.5,"CorrienteDeSalida":1.98}
[TRANSFER] crédito restante= 42
[DISPOSITION] secuencia=7 estado=accepted tiempo=132.85 ms

[TRANSFER] secuencia=8
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 100
[TRANSFER] payload= {"PotenciaGenerada":42.9,"TemperaturaDelPanel":29.8,"VoltajeDeSalida":18.8,"CorrienteDeSalida":2.28}
[TRANSFER] crédito restante= 41
[DISPOSITION] secuencia=8 estado=accepted tiempo=135.66 ms

[TRANSFER] secuencia=9
[TRANSFER] destino= /devices/lab2-python-07/messages/events
[TRANSFER] payload_bytes= 100
[TRANSFER] payload= {"PotenciaGenerada":48.7,"TemperaturaDelPanel":30.2,"VoltajeDeSalida":18.8,"CorrienteDeSalida":2.59}
[TRANSFER] crédito restante= 40
[DISPOSITION] secuencia=9 estado=accepted tiempo=144.10 ms

[FIN] Mensajes enviados: 10, aceptados: 10
[FIN] Bytes por mensaje en el enlace: enviados=336.8 recibidos=54.9
[FIN] Resumen guardado en resumen_amqp.json
[DISCONNECT] Conexión AMQP cerrada
(.venv) lab2@Laboratorio-2:~/laboratorio4$ |
```

Figura A3. Final de la corrida AMQP: 10 mensajes aceptados y bytes por mensaje.

En Datos sin procesar la serie aparece entre los eventos Dispositivo conectado (11:09:51) y Dispositivo desconectado (11:10:36), con los mismos valores de la terminal y la hora de creación que envía el cliente.

Marca de tiempo	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del Panel
> 2/10/2026, 11:10:36	Dispositivo desconectado				
> 2/10/2026, 11:10:36	Telemetría	2/10/2026, 11:10:36	2.59	48.7	30.2
> 2/10/2026, 11:10:31	Telemetría	2/10/2026, 11:10:31	2.28	42.9	29.8
> 2/10/2026, 11:10:26	Telemetría	2/10/2026, 11:10:26	1.98	36.6	28.9
> 2/10/2026, 11:10:21	Telemetría	2/10/2026, 11:10:21	1.7	31.3	28.2
> 2/10/2026, 11:10:16	Telemetría	2/10/2026, 11:10:16	1.53	28	27.9
> 2/10/2026, 11:10:11	Telemetría	2/10/2026, 11:10:11	1.37	24.9	27.2
> 2/10/2026, 11:10:06	Telemetría	2/10/2026, 11:10:06	1.16	21.1	27.1
> 2/10/2026, 11:10:01	Telemetría	2/10/2026, 11:10:01	1.1	19.8	26.7
> 2/10/2026, 11:09:56	Telemetría	2/10/2026, 11:09:56	1.06	19	26.4
> 2/10/2026, 11:09:51	Telemetría	2/10/2026, 11:09:51	1.01	18.1	26.4
> 2/10/2026, 11:09:51	Dispositivo conectado				

Figura A4. Serie AMQP en Datos sin procesar de Lab 2 Python 07.

El JSON muestra `_eventcreationtime`, que pone el cliente, y `_timestamp`, que asigna IoT Hub. En este mensaje la diferencia fue de 105 ms.

Marca de tiempo	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del Panel
> 2/10/2026, 11:10:36	Dispositivo desconectado				
> 2/10/2026, 11:10:36	Telemetría	2/10/2026, 11:10:36	2.59	48.7	30.2
<pre>1 { 2 "_eventcreationtime": "2026-10-02T16:10:36.252Z", 3 "PotenciaGenerada": 48.7, 4 "TemperaturaDelPanel": 30.2, 5 "VoltajeDeSalida": 18.8, 6 "CorrienteDeSalida": 2.59, 7 "_eventtype": "Telemetría", 8 "_timestamp": "2026-10-02T16:10:36.357Z" 9 }</pre>					
> 2/10/2026, 11:10:31	Telemetría	2/10/2026, 11:10:31	2.28	42.9	29.8
> 2/10/2026, 11:10:26	Telemetría	2/10/2026, 11:10:26	1.98	36.6	28.9

Figura A5. JSON de un mensaje AMQP con las cuatro variables y las dos marcas de tiempo.

La traza de Proton deja ver el protocolo completo: SASL PLAIN con el token oculto, open con el hostname del hub, begin (sesión), attach del enlace telemetria-panel, flow con link-credit=0x32 (50 mensajes), cada transfer de 287 bytes y su disposition accepted ya liquidada. IoT Hub anuncia además un idle-time-out de 240 s.

Figura A6. Tramas AMQP de una corrida de dos mensajes.

Primero se envió con una sola conexión reutilizada. El saludo TLS tomó 129 ms y 611/3524 bytes; después cada POST recibió 204 No Content entre 131 y 201 ms.

Figura A7. Inicio de la corrida HTTPS con keep-alive.

```
[POST] payload_bytes= 100
[POST] payload= {"PotenciaGenerada":21.8,"TemperaturaDelPanel":27.0,"VoltajeDeSalida":18.0,"CorrienteDeSalida":1.21}
[RESPUESTA] HTTP 204 No Content tiempo=140.47 ms

[POST] secuencia=4
[POST] payload_bytes= 100
[POST] payload= {"PotenciaGenerada":25.1,"TemperaturaDelPanel":27.2,"VoltajeDeSalida":18.3,"CorrienteDeSalida":1.37}
[RESPUESTA] HTTP 204 No Content tiempo=135.97 ms

[POST] secuencia=5
[POST] payload_bytes= 99
[POST] payload= {"PotenciaGenerada":29.1,"TemperaturaDelPanel":28.2,"VoltajeDeSalida":18.2,"CorrienteDeSalida":1.6}
[RESPUESTA] HTTP 204 No Content tiempo=133.55 ms

[POST] secuencia=6
[POST] payload_bytes= 100
[POST] payload= {"PotenciaGenerada":32.9,"TemperaturaDelPanel":28.3,"VoltajeDeSalida":18.5,"CorrienteDeSalida":1.78}
[RESPUESTA] HTTP 204 No Content tiempo=144.81 ms

[POST] secuencia=7
[POST] payload_bytes= 100
[POST] payload= {"PotenciaGenerada":38.9,"TemperaturaDelPanel":29.1,"VoltajeDeSalida":18.6,"CorrienteDeSalida":2.09}
[RESPUESTA] HTTP 204 No Content tiempo=138.55 ms

[POST] secuencia=8
[POST] payload_bytes= 100
[POST] payload= {"PotenciaGenerada":44.5,"TemperaturaDelPanel":29.8,"VoltajeDeSalida":18.7,"CorrienteDeSalida":2.38}
[RESPUESTA] HTTP 204 No Content tiempo=137.41 ms

[POST] secuencia=9
[POST] payload_bytes= 100
[POST] payload= {"PotenciaGenerada":50.5,"TemperaturaDelPanel":30.4,"VoltajeDeSalida":18.9,"CorrienteDeSalida":2.67}
[RESPUESTA] HTTP 204 No Content tiempo=130.93 ms

[FIN] Mensajes enviados: 10, respondidos con 204: 10
[FIN] Conexiones TLS abiertas: 1
[FIN] Bytes por mensaje (petición y respuesta): enviados=765.8 recibidos=93.0
[FIN] Bytes por mensaje contando el saludo TLS: enviados=826.9 recibidos=445.4
[FIN] Resumen guardado en resumen_https.json
(.venv) lab2@Laboratorio-2:~/laboratorio4$ |
```

Figura A8. Final de la corrida HTTPS con keep-alive: 10 respuestas 204 y bytes por mensaje.

Luego se repitió abriendo una conexión TLS nueva para cada mensaje, como haría un datalogger que despierta, envía y se apaga. Cada envío pagó un saludo de 125 a 146 ms y el costo por mensaje subió a 1379/3617 bytes.

```
[RESPUESTA] HTTP 204 No Content tiempo=180.07 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 138.18 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=2
[RESPUESTA] HTTP 204 No Content tiempo=177.29 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 129.28 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=3
[RESPUESTA] HTTP 204 No Content tiempo=202.37 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 132.22 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=4
[RESPUESTA] HTTP 204 No Content tiempo=196.91 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 124.87 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=5
[RESPUESTA] HTTP 204 No Content tiempo=199.73 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 133.81 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=6
[RESPUESTA] HTTP 204 No Content tiempo=181.83 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 146.01 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=7
[RESPUESTA] HTTP 204 No Content tiempo=204.49 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 132.82 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=8
[RESPUESTA] HTTP 204 No Content tiempo=206.11 ms
[TLS] Conexión establecida con 20.44.8.231:443 en 132.04 ms (bytes TLS: enviados=611 recibidos=3524)

[POST] secuencia=9
[RESPUESTA] HTTP 204 No Content tiempo=192.55 ms

[FIN] Mensajes enviados: 10, respondidos con 204: 10
[FIN] Conexiones TLS abiertas: 10
[FIN] Bytes por mensaje (petición y respuesta): enviados=768.0 recibidos=93.0
[FIN] Bytes por mensaje contando el saludo TLS: enviados=1379.0 recibidos=3617.0
[FIN] Resumen guardado en resumen_https_nueva.json
(.venv) lab2@Laboratorio-2:~/laboratorio4$ |
```

Figura A9. Corrida HTTPS con una conexión nueva por mensaje.

Las dos series llegaron a la misma aplicación: keep-alive de 11:17:17 a 11:18:03 y conexión nueva de 11:18:24 a 11:19:12. Como HTTPS no mantiene sesión, la serie no empieza con un evento Dispositivo conectado.

Marca de tiempo ↓	Tipo de mensaje	Hora de creación del evento	Corriente de Salida	Potencia Generada	Temperatura del Panel
> 2/10/2026, 11:19:12	Telemetría	2/10/2026, 11:19:11	2.75	52.5	31.1
> 2/10/2026, 11:19:06	Telemetría	2/10/2026, 11:19:06	2.44	45.6	30.3
> 2/10/2026, 11:19:01	Telemetría	2/10/2026, 11:19:01	2.07	38.5	29.2
> 2/10/2026, 11:18:56	Telemetría	2/10/2026, 11:18:55	1.83	33.9	28.4
> 2/10/2026, 11:18:50	Telemetría	2/10/2026, 11:18:50	1.63	30	28.1
> 2/10/2026, 11:18:45	Telemetría	2/10/2026, 11:18:45	1.43	26	27.7
> 2/10/2026, 11:18:40	Telemetría	2/10/2026, 11:18:39	1.25	22.8	27.1
> 2/10/2026, 11:18:39	Dispositivo desconectado				
> 2/10/2026, 11:18:34	Telemetría	2/10/2026, 11:18:34	1.14	20.5	26.5
> 2/10/2026, 11:18:29	Telemetría	2/10/2026, 11:18:29	1.01	18.2	26.6
> 2/10/2026, 11:18:24	Telemetría	2/10/2026, 11:18:23	0.96	17.2	26.3
> 2/10/2026, 11:18:03	Telemetría	2/10/2026, 11:18:03	2.67	50.5	30.4
> 2/10/2026, 11:17:58	Telemetría	2/10/2026, 11:17:58	2.38	44.5	29.8
> 2/10/2026, 11:17:53	Telemetría	2/10/2026, 11:17:53	2.09	38.9	29.1
> 2/10/2026, 11:17:48	Telemetría	2/10/2026, 11:17:48	1.78	32.9	28.3
> 2/10/2026, 11:17:43	Telemetría	2/10/2026, 11:17:43	1.6	29.1	28.2
> 2/10/2026, 11:17:38	Telemetría	2/10/2026, 11:17:38	1.37	25.1	27.2
> 2/10/2026, 11:17:33	Telemetría	2/10/2026, 11:17:32	1.21	21.8	27
> 2/10/2026, 11:17:27	Telemetría	2/10/2026, 11:17:27	1.12	20.2	26.7
> 2/10/2026, 11:17:22	Telemetría	2/10/2026, 11:17:22	1.06	19.2	26.4
> 2/10/2026, 11:17:17	Telemetría	2/10/2026, 11:17:17	0.97	17.6	26.6

Figura A10. Series HTTPS en Datos sin procesar.

A.5 Línea base MQTT

Para comparar en las mismas condiciones se midió MQTT desde la VM con el mismo generador y el mismo método: QoS 1, topic de eventos con las propiedades de contenido y hora de creación, sesión lista en 203 ms.

```
(.venv) lab2@Laboratorio-2:~/laboratorio4$ python mqtt_cliente.py --count 10 --interval 5
===== MQTT =====
Hostname: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net
Puerto: 8883 (mqttps)
TLS: Si, certificado validado con certifi
Username: iotc-bc967d9e-3fc7-45ff-a3a8-f0a973a03e93.azure-devices.net/lab2-python-07/?api-version=2021-04-12
Topic: devices/lab2-python-07/messages/events/
QoS: 1
Clave: Cargada, no mostrada
=====
[CONNECT] Código de respuesta: Success
[CONNECT] Sesión MQTT lista en 202.63 ms (bytes TLS + CONNECT: enviados=963 recibidos=3620)

[PUBLISH] secuencia=0
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":18.5,"TemperaturaDelPanel":26.3,"VoltajeDeSalida":18.0,"CorrienteDeSalida":1.03}
[PUBACK] secuencia=0 mid=1 tiempo=140.94 ms

[PUBLISH] secuencia=1
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":18.4,"TemperaturaDelPanel":26.6,"VoltajeDeSalida":17.9,"CorrienteDeSalida":1.03}
[PUBACK] secuencia=1 mid=2 tiempo=139.21 ms

[PUBLISH] secuencia=2
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":20.1,"TemperaturaDelPanel":26.9,"VoltajeDeSalida":18.1,"CorrienteDeSalida":1.11}
[PUBACK] secuencia=2 mid=3 tiempo=144.68 ms
|
```

Figura A11. Inicio de la corrida MQTT.

```
[PUBLISH] secuencia=3
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":22.0,"TemperaturaDelPanel":27.0,"VoltajeDeSalida":18.2,"CorrienteDeSalida":1.21}
[PUBACK] secuencia=3 mid=4 tiempo=133.46 ms

[PUBLISH] secuencia=4
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":24.0,"TemperaturaDelPanel":27.3,"VoltajeDeSalida":18.2,"CorrienteDeSalida":1.32}
[PUBACK] secuencia=4 mid=5 tiempo=141.00 ms

[PUBLISH] secuencia=5
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":29.1,"TemperaturaDelPanel":27.9,"VoltajeDeSalida":18.3,"CorrienteDeSalida":1.59}
[PUBACK] secuencia=5 mid=6 tiempo=139.74 ms

[PUBLISH] secuencia=6
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":32.9,"TemperaturaDelPanel":28.2,"VoltajeDeSalida":18.5,"CorrienteDeSalida":1.78}
[PUBACK] secuencia=6 mid=7 tiempo=150.07 ms

[PUBLISH] secuencia=7
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":38.1,"TemperaturaDelPanel":29.1,"VoltajeDeSalida":18.5,"CorrienteDeSalida":2.06}
[PUBACK] secuencia=7 mid=8 tiempo=137.06 ms

[PUBLISH] secuencia=8
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":44.6,"TemperaturaDelPanel":30.0,"VoltajeDeSalida":18.8,"CorrienteDeSalida":2.37}
[PUBACK] secuencia=8 mid=9 tiempo=139.72 ms

[PUBLISH] secuencia=9
[PUBLISH] payload_bytes= 100
[PUBLISH] payload= {"PotenciaGenerada":49.9,"TemperaturaDelPanel":30.7,"VoltajeDeSalida":18.9,"CorrienteDeSalida":2.64}
[PUBACK] secuencia=9 mid=10 tiempo=144.87 ms

[FIN] Mensajes enviados: 10, confirmados con PUBACK: 10
[FIN] Bytes por mensaje: enviados=263.0 recibidos=33.0
[FIN] Resumen guardado en resumen_mqtt.json
(.venv) lab2@Laboratorio-2:~/laboratorio4$ |
```

Figura A12. Final de la corrida MQTT: 10 PUBACK y bytes por mensaje.

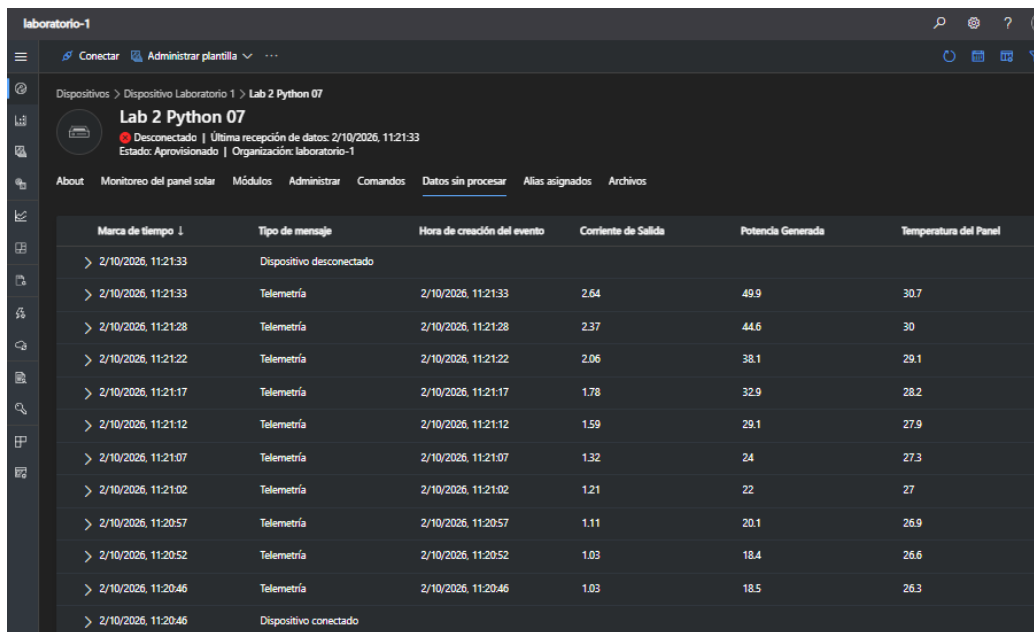


Figura A13. Serie MQTT en Datos sin procesar entre los eventos de conexión y desconexión.

A.6 Comparación y Explorador de datos

resumen.py reúne los resultados de las cuatro series a partir de los archivos de cada cliente.

```
(.venv) lab2@Laboratorio-2:~/Laboratorio4$ python resumen.py
Protocolo      OK      Conexión ms  Bytes conexión env/rec  Bytes por mensaje env/rec  Confirmación ms min/prom/max
-----
MQTT 8883      10/10    203          963/3620                263/33                    133/141/150
AMQP 5671      10/10    285          1216/4200                337/55                    133/142/171
HTTPS keep-alive 10/10    129          611/3524                766/93                    131/144/201
HTTPS conexión nueva 10/10    135          611/3524                1379/3617                 136/188/206

Payload JSON en los tres casos: mismas 4 variables, 99 a 101 bytes.
Bytes medidos con ss (carga TCP, incluye TLS). En HTTPS con conexión nueva el saludo TLS va en cada mensaje.
(.venv) lab2@Laboratorio-2:~/Laboratorio4$
```

Figura A14. Tabla comparativa generada en la VM.

Con las marcas de tiempo de IoT Central se calculó, mensaje por mensaje, el tiempo entre la creación en el cliente y el registro en IoT Hub:

Protocolo	Mensajes	Mínimo	Promedio	Máximo
MQTT	10	97 ms	107 ms	115 ms
AMQP	10	94 ms	109 ms	142 ms
HTTPS keep-alive	10	103 ms	114 ms	173 ms
HTTPS conexión nueva	10	107 ms	155 ms	178 ms

En HTTPS con conexión nueva la hora de creación se toma después del saludo TLS, así que esos 155 ms no incluyen los 125 a 146 ms del saludo. Por último, el Explorador de datos muestra la Potencia Generada de Lab 2 Python 07 en los últimos 30 minutos, con una subida por cada serie: AMQP a las 11:10, HTTPS entre 11:17 y 11:19 y MQTT a las 11:21.

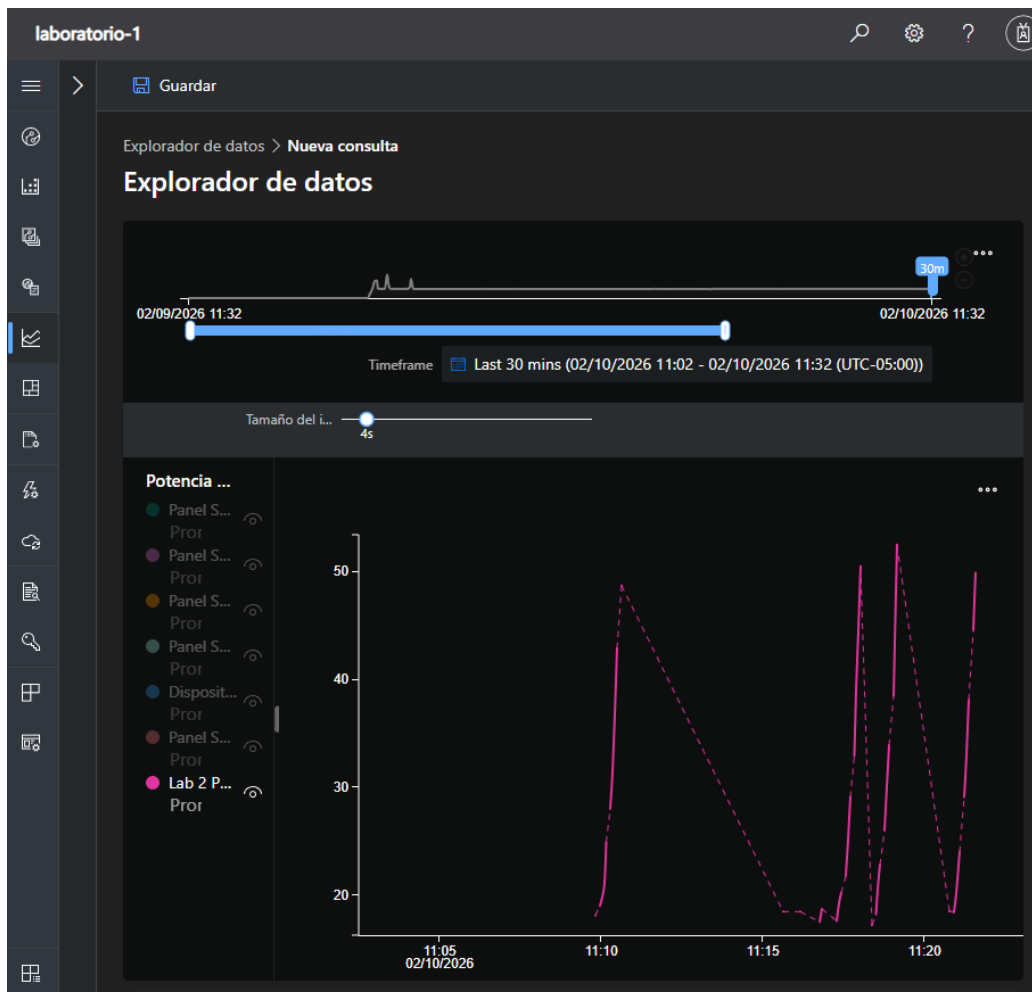


Figura A15. Potencia Generada de Lab 2 Python 07 en el Explorador de datos.

Repositorio GitHub

<https://github.com/Hypnostian/IOT.git>